Министерство образования Республики Беларусь
Учреждение образования «Белорусский государственный университет информатики и радиоэлектроники»

Факультет компьютерных систем и сетей
Кафедра Информатики
Дисциплина «Операционные среды и системное программирование»

ОТЧЕТ
к лабораторной работе №4
на тему:
«ВЗАИМОДЕЙСТВИЕ ПРОЦЕССОВ (ПОТОКОВ): ВЗАИМНОЕ ИСКЛЮЧЕНИЕ И СИНХРОНИЗАЦИЯ»
БГУИР 6-05-0612-02 47

Выполнила студентка группы 453504
ЖУКОВСКАЯ Мария Александровна

_____
(дата, подпись студента)

Проверил ст. преп. каф. Информатики
СЫС Анна Дмитриевна

_____
(дата, подпись преподавателя)

Минск 2026

ОГЛАВЛЕНИЕ

Введение ... 3
1 Цель работы ... 4
2 Задание ... 4
3 Выполнение работы ... 4
3.1 Архитектура разработанной программы ... 4
3.2 Описание процессов, потоков и общих ресурсов ... 5
3.3 Описание примененных механизмов синхронизации ... 6
3.4 Алгоритм работы программы ... 6
3.5 Схемы и диаграммы взаимодействия ... 7
3.6 Перечень использованных функций Windows API ... 7
3.7 Описание проверяемых инвариантов ... 8
3.9 Методика проведения экспериментов ... 9
4 Результаты выполнения ... 9
4.1 Экспериментальные результаты ... 9
4.2 Графики и анализ результатов ... 10
Заключение ... 13
Список использованных источников ... 14
Приложение А. Сведения о репозитории Git ... 15
Приложение Б. Листинг кода генерации графических материалов ... 16
Приложение В. Листинг кода исходного задания ... 19

ВВЕДЕНИЕ

Современные операционные системы обеспечивают параллельное выполнение множества потоков, которые нередко обращаются к общим ресурсам. При отсутствии согласованного доступа возникают состояния гонки, приводящие к некорректным результатам и трудно воспроизводимым ошибкам. Для их предотвращения ОС предоставляет механизмы синхронизации – мьютексы, семафоры, события, критические секции. Выбор подходящего механизма и корректная организация взаимодействия потоков влияют как на правильность работы программы, так и на её производительность.

Настоящая лабораторная работа посвящена изучению механизмов взаимного исключения и синхронизации в Windows. В качестве модели рассматривается классическая задача «Писатели–читатели» с приоритетом писателей: наличие ожидающего писателя блокирует начало новых операций чтения, при этом уже выполняющиеся чтения должны корректно завершиться. Для синхронизации используются мьютексы (Mutex) и семафоры (Semaphore) Windows API.

Программа реализована на C++17, собирается MSVC в Visual Studio 2022 и запускается в Windows 10/11 через терминал Developer PowerShell for VS 2022. Экспериментальные данные обрабатываются на Python с библиотекой Matplotlib, проект ведётся в Git с разделением на четыре контрольные точки.

1 ЦЕЛЬ РАБОТЫ

Изучение механизмов взаимного исключения и синхронизации в операционной системе Windows, приобретение практических навыков организации согласованной параллельной работы процессов и потоков.

2 ЗАДАНИЕ

Задание 1. Вариант 10. Реализовать алгоритм, при котором наличие ожидающего писателя препятствует началу новых операций чтения. Уже выполняющиеся чтения должны завершиться. Исследовать: 3 читателя и 2 писателя, 3 читателя и 5 писателей. Определить время ожидания каждой категории участников.

3 ВЫПОЛНЕНИЕ РАБОТЫ

3.1 Архитектура разработанной программы

Логически программу можно разделить на пять слоёв:

1 Слой конфигурации — структура Config и функция parseArgs — отвечает за хранение и разбор параметров модели: числа читателей и писателей, количества операций на участника, длительности чтения и записи, тайм-аута ожидания и пути к журналу. Все параметры задаются через аргументы командной строки и имеют значения по умолчанию, поэтому поведение программы меняется без правки исходного кода, как и требуется в задании.

2 Слой участников — функции reader и writer, выполняющиеся в рабочих потоках. Каждый читатель и каждый писатель выполняет заданное число операций; в каждой операции участник проходит состояния ожидания, захвата ресурса, работы с ним и освобождения.

3 Слой синхронизации включает четыре объекта. Семафор g_resource с начальным значением 1 обеспечивает доступ к разделяемым данным: его захватывает первый читатель или писатель, а освобождает последний читатель или писатель. Семафор g_readTry реализует приоритет писателей: первый ожидающий писатель захватывает его и блокирует вход новых читателей, а возвращается шлагбаум только после завершения всех писателей. Мьютексы g_rmutex и g_wmutex защищают счётчики зарегистрированных читателей и писателей.

4 Слой журналирования и статистики — функции logEvent, waitOk, addWait и структура PStats. Журнал защищён критической секцией g_logCs, поскольку его пишут одновременно несколько потоков; критическая секция выбрана как самый лёгкий механизм для коротких захватов внутри одного процесса. Функция waitOk оборачивает WaitForSingleObject и различает успех, тайм-аут и ошибку, учитывая их в разных счётчиках. Каждый участник

пишет статистику только в свою запись PStats, поэтому гонки за статистику не возникает.

В модели один разделяемый ресурс — база данных, к которой читатели обращаются параллельно, а писатели эксклюзивно. Сам ресурс не моделируется: его роль играет семафор g_resource, а работа с ним — Sleep. В рантайме проверяются три инварианта: читатель не входит при активном писателе, писатель не входит, пока активен хотя бы один читатель или другой писатель, и после завершения всех потоков не остаётся удерживаемых ресурсов и ненулевых счётчиков. Нарушения фиксируются счётчиком g_violations, который в корректной реализации всегда равен нулю. Дополнительно waitOk различает тайм-ауты и ошибки ожидания, что позволяет отделить естественное для политики writers-preference голодание читателей от действительных сбоев синхронизации.

Обработка ошибок включает проверку результатов создания объектов синхронизации и открытия журнала, различение тайм-аутов и ошибок ожидания, откат уже захваченных ресурсов при неудачном ожидании и освобождение всех дескрипторов и критических секций в конце работы. После join проверяется, что не осталось активных участников и ненулевых счётчиков.

3.2 Описание процессов, потоков и общих ресурсов

Программа реализована на C++17 в одном файле main.cpp, запускается как один процесс. Работа ведётся в Visual Studio Code: код редактируется в редакторе, сборка и запуск — из терминала Developer PowerShell for VS 2022. Проект организован по стандартной схеме: исходный код в src/, тесты в tests/, экспериментальные данные и графики в data/, скрипт обработки в scripts/, схема в diagrams/, отчёт в report/. Разделение на модули не делается: весь код занимает около 350 строк, и вынесение логгера или статистики в отдельные файлы усложнило бы сборку без выигрыша в читаемости.

Внутри процесса создаются рабочие потоки двух типов — читатели и писатели, их число задаётся параметрами --readers и --writers. Каждый поток имеет уникальный номер: читатели получают номера от 1 до readers, писатели продолжают нумерацию. Главный поток выполняет main: разбирает аргументы, инициализирует таймер, критическую секцию, журнал и объекты синхронизации, создаёт рабочие потоки, ждёт их через join, проверяет инварианты, агрегирует статистику и выводит сводку. Рабочие потоки выполняют reader и writer.

Общие ресурсы — счётчики g_readCount и g_writeCount (число зарегистрированных читателей и писателей), атомарные g_activeReaders и g_activeWriters (число участников в критической секции, нужны для проверки инвариантов) и сам разделяемый ресурс, роль которого играет семафор g_resource. Каждый поток пишет статистику только в свою запись PStats, поэтому гонки за статистику нет. Журнал — общий ресурс, его пишут все потоки, поэтому он защищён критической секцией g_logCs.

3.3 Описание примененных механизмов синхронизации

Используются четыре объекта синхронизации и одна критическая секция:

1 Семафор g_resource (начальное значение 1) обеспечивает взаимное исключение доступа к данным. Захватывает его первый читатель или писатель, освобождает последний — это разные потоки, поэтому мьютекс не подходит, а семафор подходит.

2 Семафор g_readTry (начальное значение 1) реализует приоритет писателей. Первый ожидающий писатель захватывает его и блокирует вход новых читателей; уже читающие дочитывают, так как прошли шлагбаум раньше. Возвращается шлагбаум, только когда последний писатель покидает систему (g_writeCount == 0). Здесь снова нужен семафор: захватывает первый писатель, освобождает последний.

3 Мьютексы g_rmutex и g_wmutex защищают счётчики. Без g_rmutex два читателя могли бы одновременно увидеть g_readCount == 0, оба решить, что они первые, и оба ждать g_resource; один получил бы ресурс, второй — тайм-аут. Мьютекс делает проверку и инкремент атомарными. То же для g_wmutex.

4 Критическая секция g_logCs защищает журнал: это самый лёгкий механизм для коротких захватов внутри одного процесса, и захват с освобождением выполняются одним потоком в logEvent.

Все объекты безымянные, так как синхронизируются потоки одного процесса. Взаимных блокировок нет: порядок захвата всегда одинаков (мьютекс счётчика, затем семафор ресурса), а g_readTry захватывает только первый писатель. Голодание читателей при большом числе писателей — ожидаемое следствие политики, а не дефект.

3.4 Алгоритм работы программы

– главный поток разбирает аргументы, инициализирует таймер, критическую секцию, журнал и объекты синхронизации, создаёт массив stats и вектор threads, запускает читателей, затем писателей, после чего ждёт их через join;

– читатель в каждой итерации ждёт g_readTry с тайм-аутом, при неудаче увеличивает счётчик тайм-аутов и переходит к следующей операции;

– получив шлагбаум, читатель захватывает g_rmutex, увеличивает g_readCount и определяет, первый ли он, и, если первый, дополнительно ждёт g_resource, при неудаче откатывает изменения;

– затем читатель отпускает g_rmutex, возвращает g_readTry, проверяет отсутствие активных писателей и увеличивает g_activeReaders;

– после Sleep(readMs) уменьшает g_activeReaders, снова захватывает g_rmutex, уменьшает g_readCount и, если он последний, освобождает g_resource;

– писатель в каждой итерации захватывает g_wmutex, увеличивает g_writeCount и определяет, первый ли он, и, если первый, захватывает g_readTry, при неудаче откатывает изменения;

– затем отпускает g_wmutex и ждёт g_resource с тайм-аутом, при неудаче снимает регистрацию под g_wmutex и, если он последний, возвращает g_readTry;

– получив ресурс, писатель проверяет отсутствие активных читателей и писателей, увеличивает g_activeWriters, выполняет Sleep(writeMs), уменьшает g_activeWriters, освобождает g_resource и снимает регистрацию под g_wmutex, возвращая g_readTry, если он последний;

– после join главный поток проверяет, что все счётчики равны нулю, фиксирует общее время, агрегирует статистику по категориям, печатает сводку и освобождает ресурсы.

3.5 Схемы и диаграммы взаимодействия

Схема взаимодействия участников приведена на рисунке 3.1.

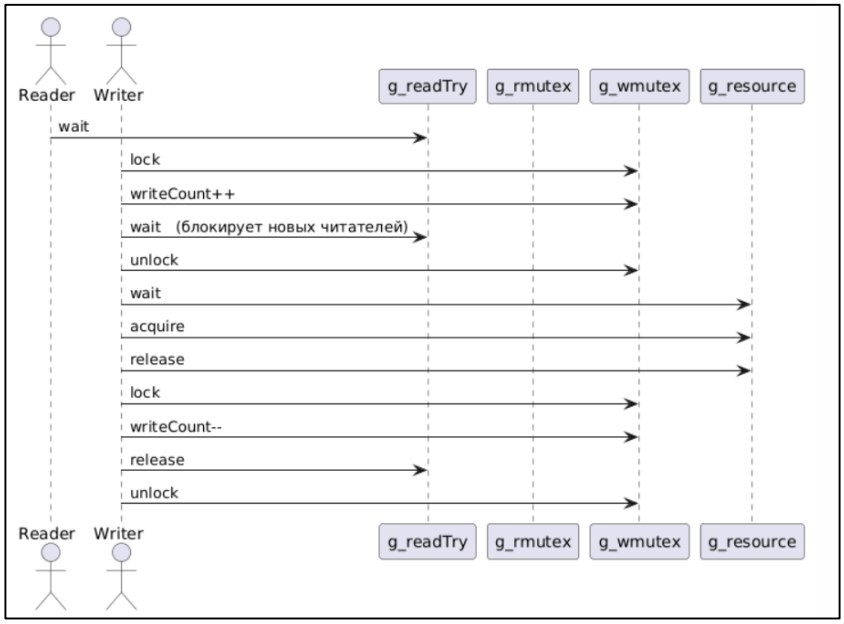


Рисунок 3.1 – Схема взаимодействия участников

3.6 Перечень использованных функций Windows API

– CreateMutex — создание мьютексов g_rmutex и g_wmutex для защиты счётчиков читателей и писателей;

– CreateSemaphore — создание семафоров g_resource (доступ к данным) и g_readTry (шлагбаум приоритета писателей), оба с начальным значением 1 и максимумом 1;

– WaitForSingleObject — ожидание объекта синхронизации с тайм-аутом или INFINITE; используется через обёртку waitOk, которая различает WAIT_OBJECT_0, WAIT_TIMEOUT и ошибку;

– ReleaseMutex — освобождение мьютекса счётчика после его изменения;

– ReleaseSemaphore — освобождение семафора: возврат g_readTry первым/последним писателем, освобождение g_resource последним читателем или писателем;

– CloseHandle — закрытие всех хендлов объектов синхронизации в конце работы;

– InitializeCriticalSection / DeleteCriticalSection — инициализация и удаление критической секции g_logCs, защищающей журнал;

– EnterCriticalSection / LeaveCriticalSection — вход и выход из критической секции при записи события в журнал;

– QueryPerformanceFrequency / QueryPerformanceCounter — высокоточный таймер для измерения времени ожидания и общего времени выполнения;

– Sleep — имитация длительности операций чтения и записи, а также стартовых задержек; используется только для моделирования, не для синхронизации;

– GetLastError — получение кода ошибки при неудачном создании объектов синхронизации или открытии журнала.

3.7 Описание проверяемых инвариантов

1 Взаимное исключение чтения и записи. Читатель не входит в критическую секцию при активном писателе — проверяется в reader по g_activeWriters > 0 сразу после входа.

2 Эксклюзивность записи. Писатель не входит, пока активен хотя бы один читатель или другой писатель — проверяется в writer по g_activeReaders > 0 || g_activeWriters > 0.

3 Отсутствие удерживаемых ресурсов после завершения. После join все счётчики g_readCount, g_writeCount, g_activeReaders, g_activeWriters должны быть нулевыми — проверяется в main.

4 Согласованность счётчиков: g_readCount и g_writeCount изменяются только под g_rmutex и g_wmutex, поэтому промежуточных «полусостояний» не бывает.

5 Корректность захвата и освобождения. g_readTry захватывает первый писатель, а возвращает последний (g_writeCount == 0), поэтому шлагбаум не может остаться захваченным после ухода всех писателей.

6 Любое нарушение увеличивает счётчик g_violations.

3.9 Методика проведения экспериментов

Исследуемые конфигурации: 3 читателя / 2 писателя и 3 читателя / 5 писателей; изменяемый параметр — число писателей, остальные фиксированы (20 операций на участника, 50 мс на чтение и запись, тайм-аут 5000 мс). Для каждой конфигурации выполняется не менее трёх запусков подряд без изменения кода и параметров

Запуск выполняется из терминала Developer PowerShell for VS 2022 с флагом --quiet, чтобы отключить журнал и не искажать замеры операциями ввода-вывода.

Вывод каждого запуска сохраняется в data/run_<конфигурация>_<номер>.txt для последующей обработки и воспроизводимости. Из каждого запуска фиксируются: общее время выполнения, число успешных операций, среднее и максимальное время ожидания по категориям, количество тайм-аутов, число нарушений инвариантов.

Скрипт scripts/plot.py парсит сохранённые файлы, собирает data/summary.csv со средними по трём запускам и строит графики: общее время выполнения от числа писателей, среднее время ожидания по категориям, количество тайм-аутов.

4 РЕЗУЛЬТАТЫ ВЫПОЛНЕНИЯ

4.1 Экспериментальные результаты

Результаты эксперимента приведены в таблице 4.1.

Таблица 4.1 – Результаты эксперимента

Конфигурация	Запуск 1, мс	Запуск 2, мс	Запуск 3, мс	Среднее, мс
А: 3 читателя / 2 писателя	3801	3791	3798	3797
Б: 3 читателя / 5 писателя	7473	7481	7473	7476

Вывод программы при запуске конфигурации А представлен на рисунке 4.1.

```
PS C:\ddd\Unik\5sem\osisp\lr4> .\build\main.exe --readers 3 --writers 2 --ops 20 --read-ms 50 --write-ms 50 --quiet | Te
e-Object data\run_3r2w_1.txt
STUDENT_ID = 123 | SEED = 12931 | OPERATIONS = 635
=== SUMMARY ===
Readers: ops=60 avg_wait_ms=126 max_wait_ms=2554 timeouts=0
Writers: ops=40 avg_wait_ms=63 max_wait_ms=123 timeouts=0
Total time, ms: 3801
Invariant violations: 0
Timeouts total: 0
```

Рисунок 4.1 – Вывод программы при запуске конфигурации А

Вывод программы при запуске конфигурации Б представлен на рисунке 4.2.

```
PS C:\ddd\Unik\5sem\osisp\lr4> .\build\main.exe --readers 3 --writers 5 --ops 20 --read-ms 50 --write-ms 50 --quiet | Te
e-Object data\run_3r5w_1.txt
STUDENT_ID = 123 | SEED = 12931 | OPERATIONS = 635
=== SUMMARY ===
Readers: ops=57 avg_wait_ms=66 max_wait_ms=1277 timeouts=3
Writers: ops=100 avg_wait_ms=245 max_wait_ms=311 timeouts=0
Total time, ms: 7473
Invariant violations: 0
Timeouts total: 3
```

Рисунок 4.2 – Вывод программы при запуске конфигурации Б

Профильные показатели приведены в таблице 4.2.

Таблица 4.2 – Профильные показатели

Конфигурация	Категория	Успешных операций	Тайм-ауты	Ср. ожидание, мс	Макс. ожидание, мс	Нарушений
3R / 2W	Читатели	60	0	126	2548	0
3R / 2W	Писатели	40	0	63	119	0
3R / 5W	Читатели	57	3	66	1284	0
3R / 5W	Писатели	100	0	245	310	0

4.2 Графики и анализ результатов

График количества тайм-аутов по категориям показан на рисунке 4.3.

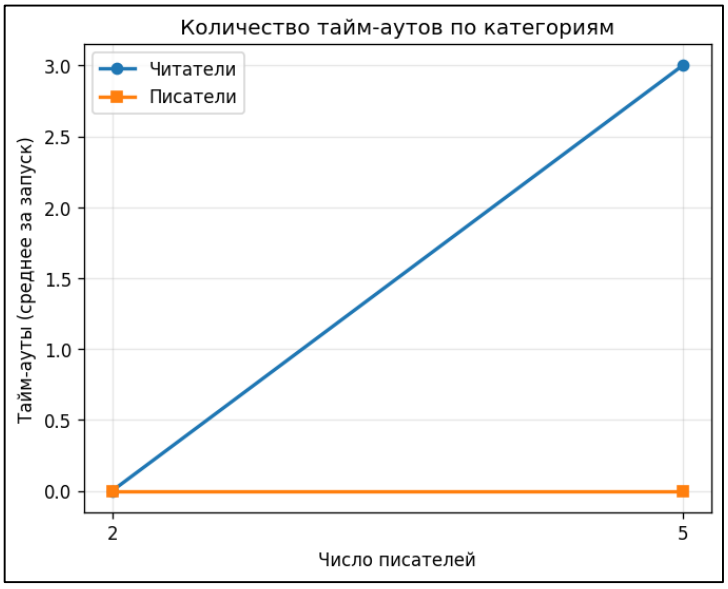


Рисунок 4.3 – График количества тайм-аутов по категориям

Зависимость времени выполнения от числа писателей приведена на рисунке 4.4.

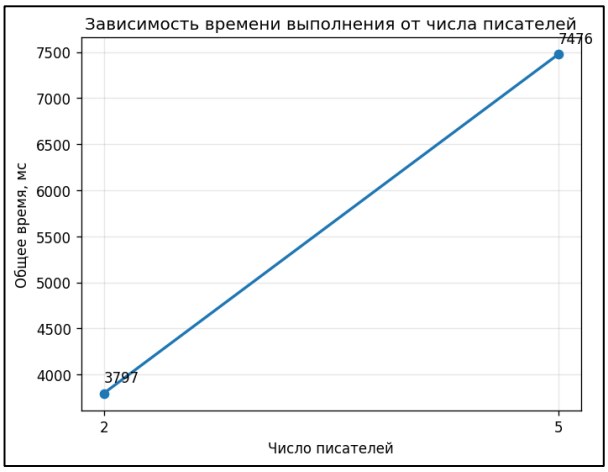


Рисунок 4.4 – График времени выполнения от числа писателей

График времени ожидания по категориям участников приведена на рисунке 4.5.

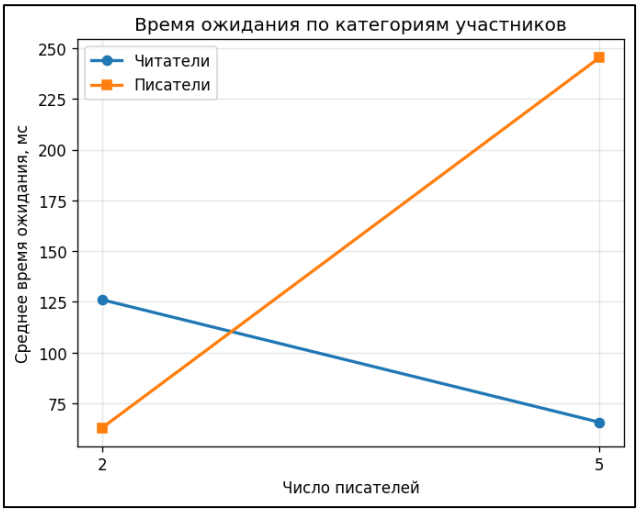


Рисунок 4.5 – График времени ожидания по категориям участников

Ожидалось, что при росте числа писателей вырастет их ожидание из-за сериализации на g_resource, читатели будут ждать дольше из-за g_readTry, при 5 писателях появятся тайм-ауты у читателей, а общее время вырастет непропорционально нагрузке.

По результатам время выросло с 3797 до 7476 мс, то есть в 1.97 раза при росте числа писателей в 2.5 раза. При 3R/2W все 60 чтений и 40 записей выполнены, тайм-аутов нет. При 3R/5W успешных чтений 57: три читателя упираются в тайм-аут и уходят. Ожидание писателей выросло с 63 до 245 мс, читателей — снизилось с 126 до 66 мс. Нарушений инвариантов нет.

Семафор g_readTry реализует приоритет писателей: первый ожидающий писатель блокирует новых читателей, а шлагбаум возвращается только при g_writeCount == 0. При 5 писателях очередь писателей длится дольше, и часть читателей не успевает войти. Семафор g_resource сериализует писателей, из-

за чего их ожидание растёт в 3.9 раза. Мьютексы g_rmutex и g_wmutex обеспечивают атомарность счётчиков.

Непропорциональность объясняется тремя причинами: писатели сериализуются на g_resource, тайм-ауты сокращают фактическую нагрузку, а накладные расходы на синхронизацию растут с числом потоков. Снижение среднего ожидания читателей — артефакт метрики: статистика учитывает только успешные операции, а те читатели, которые ждали бы дольше всех, уходят в тайм-аут и в среднее не попадают.

ЗАКЛЮЧЕНИЕ

В ходе лабораторной работы изучены механизмы синхронизации потоков в Windows и реализована модель «Писатели–читатели» с приоритетом писателей на C++17 с использованием Windows API.

Программа использует семафоры g_resource и g_readTry, мьютексы g_rmutex и g_wmutex и критическую секцию g_logCs. Выбор механизмов обусловлен семантикой задачи: счётчики защищены мьютексами, поскольку захватываются и освобождаются одним потоком, а доступ к данным и шлагбаум приоритета писателей реализованы через семафоры, так как захватывает их один поток, а освобождает другой. Во всех запусках нарушений инвариантов не зафиксировано, что подтверждает корректность реализованной синхронизации.

Эксперименты для двух конфигураций проведены по три запуска каждая, результаты и их анализ приведены в соответствующих разделах отчёта. Рост нагрузки не даёт пропорционального изменения производительности: писатели сериализуются на семафоре ресурса, тайм-ауты сокращают фактическую нагрузку, а накладные расходы на синхронизацию растут с числом потоков.

Таким образом, были изучены механизмы синхронизации Windows API, модель «Писатели–читатели» с приоритетом писателей реализована корректно, влияние числа писателей на работу модели исследовано экспериментально.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

[1] Synchronization Function [Электронный ресурс]. – Режим доступа: https://learn.microsoft.com/en-us/windows/win32/sync/synchronization-functions. – Дата доступа 24.10.2026.

[2] Using Mutex Objects [Электронный ресурс]. – Режим доступа: https://learn.microsoft.com/en-us/windows/win32/sync/using-mutex-objects. – Дата доступа 24.10.2026.

[3] Using Semaphore Objects [Электронный ресурс]. – Режим доступа: https://learn.microsoft.com/en-us/windows/win32/sync/using-semaphore-objects. – Дата доступа 25.10.2026.

[4] Critical Section Objects [Электронный ресурс]. – Режим доступа: https://learn.microsoft.com/en-us/windows/win32/sync/critical-section-objects. – Дата доступа 25.10.2026.

[5] Моделирование монитора Хоара (задача читатели-писатели) [Электронный ресурс]. – Режим доступа: https://pro-prof.com/forums/topic/hoar_modelling?ysclid=mvr1z7dh2247834209. – Дата доступа 25.10.2026.

[6] Windows Internals: Thread Management — Part 2 [Электронный ресурс]. – Режим доступа https://medium.com/windows-os-internals/windows-internals-thread-management-part-2-75cfed18f9ca. – Дата доступа 25.10.2026.

[7] C++ Tutorial Multi-Threaded Programming II Native Thread for Win32 (B) (Synchronization and Resource Sharing) - 2020 [Электронный ресурс]. – Режим доступа: https://www.bogotobogo.com/cplusplus/multithreading_win32B.php#google_vignette. – Дата доступа 25.10.2026.

ПРИЛОЖЕНИЕ А. СВЕДЕНИЯ О РЕПОЗИТОРИИ GIT

Проект ведётся в Git. Репозиторий публичный:
- Платформа: GitHub
- URL: https://github.com/zhuchok0/osisp-lab4
- Ветка по умолчанию: master
- Структура: osisp4/ (src, tests, data, scripts, diagrams, report)

Номера хэшов, содержащих основную реализацию программы представлены в таблице А.1.

Таблица А.1 – Хэши и соответствующее им содержимое

Хэш коммита	Описание
8428bd6	Проектирование, структура проекта, схема
1d86ba9	Базовая реализация: потоки, состояния, журнал
216a910	Синхронизация, инварианты, обработка тайм-аутов, тесты
4478c4a	Экспериментальные данные, графики, отчёт

ПРИЛОЖЕНИЕ Б. ЛИСТИНГ КОДА ГЕНЕРАЦИИ ГРАФИЧЕСКИХ МАТЕРИАЛОВ

```
      import re
      import csv
      from pathlib import Path
      import matplotlib.pyplot as plt

      ROOT = Path(__file__).resolve().parent.parent
      DATA = ROOT / "data"

      RE_R                                                                    =
re.compile(r"Readers:\s+ops=(\d+)\s+avg_wait_ms=(\d+)\s+max_wait_ms=(\d+)\s+t
imeouts=(\d+)")
      RE_W                                                                    =
re.compile(r"Writers:\s+ops=(\d+)\s+avg_wait_ms=(\d+)\s+max_wait_ms=(\d+)\s+t
imeouts=(\d+)")
      RE_T = re.compile(r"Total time, ms:\s+(\d+)")
      RE_V = re.compile(r"Invariant violations:\s+(\d+)")


      def parse(p: Path):
          text = p.read_text(encoding="utf-16")
          r, w, t, v = RE_R.search(text), RE_W.search(text), RE_T.search(text),
RE_V.search(text)
          if not all([r, w, t, v]):
              print(f"[skip] {p.name}: cannot parse")
              return None
          return {
              "total_ms":    int(t.group(1)),
              "readers_ops": int(r.group(1)),
              "readers_avg": int(r.group(2)),
              "readers_max": int(r.group(3)),
              "readers_to":  int(r.group(4)),
              "writers_ops": int(w.group(1)),
              "writers_avg": int(w.group(2)),
              "writers_max": int(w.group(3)),
              "writers_to":  int(w.group(4)),
              "violations":  int(v.group(1)),
          }


      CONFIGS = [
          ("3R/2W", "run_3r2w_*.txt", 2),
          ("3R/5W", "run_3r5w_*.txt", 5),
      ]


      def collect():
          if not DATA.exists():
              print(f"[error] no data directory: {DATA}")
              return []
          print(f"[info] DATA = {DATA}")
          rows = []
          for label, pattern, writers in CONFIGS:
              files = sorted(DATA.glob(pattern))
              print(f"[info] {label}: {pattern} -> {len(files)} files")
              for p in files:
                  row = parse(p)
                  if row:
                      row["config"]    = label
                      row["writers_n"] = writers
```

```
                    row["run"]       = p.stem
                    rows.append(row)
        return rows


    def save_csv(rows):
        fields = ["config", "run", "writers_n", "total_ms",
                  "readers_ops", "readers_avg", "readers_max", "readers_to",
                  "writers_ops", "writers_avg", "writers_max", "writers_to",
                  "violations"]
        out = DATA / "summary.csv"
        with out.open("w", newline="", encoding="utf-8") as f:
            w = csv.DictWriter(f, fieldnames=fields)
            w.writeheader()
            for r in rows:
                w.writerow({k: r[k] for k in fields})
        print(f"[ok] wrote {out}")


    def mean(rows, key):
        return sum(r[key] for r in rows) / len(rows) if rows else 0


    def plot(rows):
        labels    = [c[0] for c in CONFIGS]
        writers_n = [c[2] for c in CONFIGS]
        by_cfg = {label: [r for r in rows if r["config"] == label] for label,
_, _ in CONFIGS}

        plt.figure()
        means = [mean(by_cfg[c], "total_ms") for c in labels]
        plt.plot(writers_n, means, marker="o", linewidth=2)
        for x, y in zip(writers_n, means):
            plt.annotate(f"{y:.0f}",  (x,  y),  textcoords="offset  points",
xytext=(0, 8))
        plt.xlabel("Число писателей")
        plt.ylabel("Общее время, мс")
        plt.title("Зависимость времени выполнения от числа писателей")
        plt.xticks(writers_n)
        plt.grid(True, alpha=0.3)
        plt.savefig(DATA       /       "plot_total_time.png",       dpi=120,
bbox_inches="tight")
        plt.close()

        plt.figure()
        r_wait = [mean(by_cfg[c], "readers_avg") for c in labels]
        w_wait = [mean(by_cfg[c], "writers_avg") for c in labels]
        plt.plot(writers_n,      r_wait,      marker="o",      linewidth=2,
label="Читатели")
        plt.plot(writers_n,      w_wait,      marker="s",      linewidth=2,
label="Писатели")
        plt.xlabel("Число писателей")
        plt.ylabel("Среднее время ожидания, мс")
        plt.title("Время ожидания по категориям участников")
        plt.xticks(writers_n)
        plt.legend()
        plt.grid(True, alpha=0.3)
        plt.savefig(DATA       /       "plot_wait_time.png",       dpi=120,
bbox_inches="tight")
        plt.close()

        plt.figure()
        r_to = [mean(by_cfg[c], "readers_to") for c in labels]
        w_to = [mean(by_cfg[c], "writers_to") for c in labels]
```

```
        plt.plot(writers_n, r_to, marker="o", linewidth=2, label="Читатели")
        plt.plot(writers_n, w_to, marker="s", linewidth=2, label="Писатели")
        plt.xlabel("Число писателей")
        plt.ylabel("Тайм-ауты (среднее за запуск)")
        plt.title("Количество тайм-аутов по категориям")
        plt.xticks(writers_n)
        plt.legend()
        plt.grid(True, alpha=0.3)
        plt.savefig(DATA / "plot_timeouts.png", dpi=120, bbox_inches="tight")
        plt.close()

        print("[ok]    wrote    plot_total_time.png,    plot_wait_time.png,
plot_timeouts.png")


    def main():
        rows = collect()
        if not rows:
            print("No run_*.txt parsed. Check that data/run_3r2w_*.txt and
data/run_3r5w_*.txt exist.")
            return
        save_csv(rows)
        plot(rows)


    if __name__ == "__main__":
        main()
```

ПРИЛОЖЕНИЕ В. ЛИСТИНГ КОДА ИСХОДНОГО ЗАДАНИЯ

```
int main(int argc, char** argv) {
    if (!parseArgs(argc, argv)) return 1;
    if (g_cfg.ops <= 0) g_cfg.ops = OPERATIONS;

    std::cout << "STUDENT_ID = " << STUDENT_ID
        << " | SEED = " << SEED
        << " | OPERATIONS = " << OPERATIONS << "\n";

    QueryPerformanceFrequency(&g_qpcFreq);
    QueryPerformanceCounter(&g_qpcStart);
    InitializeCriticalSection(&g_logCs);

    g_log.open(g_cfg.logPath);
    if (!g_log) {
        std::cerr << "Cannot open log file: " << g_cfg.logPath << "\n";
        DeleteCriticalSection(&g_logCs);
        return 1;
    }

    g_rmutex = CreateMutex(NULL, FALSE, NULL);
    g_wmutex = CreateMutex(NULL, FALSE, NULL);
    g_readTry = CreateSemaphore(NULL, 1, 1, NULL);
    g_resource = CreateSemaphore(NULL, 1, 1, NULL);

    if (!g_rmutex || !g_wmutex || !g_readTry || !g_resource) {
        std::cerr << "Create synchronization object failed: " <<
GetLastError() << "\n";
        return 1;
    }

    int total = g_cfg.readers + g_cfg.writers;
    std::vector<PStats> stats(total);
    std::vector<std::thread> threads;
    threads.reserve(total);

    int idx = 0;
    for (int i = 0; i < g_cfg.readers; ++i, ++idx) {
        stats[idx].id = idx + 1;
        stats[idx].role = "R";
        threads.emplace_back(reader, idx + 1, &stats[idx]);
    }
    for (int i = 0; i < g_cfg.writers; ++i, ++idx) {
        stats[idx].id = idx + 1;
        stats[idx].role = "W";
        threads.emplace_back(writer, idx + 1, &stats[idx]);
    }

    for (auto& t : threads) t.join();

    if (g_activeReaders.load() != 0 || g_activeWriters.load() != 0 ||
        g_readCount != 0 || g_writeCount != 0) {
        g_violations++;
    }

    long long elapsed = nowMs();

    long long rOps = 0, rWait = 0, rMax = 0;
    int rTimeout = 0;
    long long wOps = 0, wWait = 0, wMax = 0;
```

```
    int wTimeout = 0;

    for (auto& s : stats) {
        if (std::string(s.role) == "R") {
            rOps += s.ops; rWait += s.totalWait;
            rMax = std::max(rMax, s.maxWait);
            rTimeout += s.timeouts;
        }
        else {
            wOps += s.ops; wWait += s.totalWait;
            wMax = std::max(wMax, s.maxWait);
            wTimeout += s.timeouts;
        }
    }

    std::cout << "=== SUMMARY ===\n";
    std::cout << "Readers: ops=" << rOps
        << " avg_wait_ms=" << (rOps ? rWait / rOps : 0)
        << " max_wait_ms=" << rMax
        << " timeouts=" << rTimeout << "\n";
    std::cout << "Writers: ops=" << wOps
        << " avg_wait_ms=" << (wOps ? wWait / wOps : 0)
        << " max_wait_ms=" << wMax
        << " timeouts=" << wTimeout << "\n";
    std::cout << "Total time, ms: " << elapsed << "\n";
    std::cout << "Invariant violations: " << g_violations.load() << "\n";
    std::cout << "Timeouts total: " << g_timeouts.load() << "\n";

    CloseHandle(g_resource);
    CloseHandle(g_readTry);
    CloseHandle(g_wmutex);
    CloseHandle(g_rmutex);
    g_log.close();
    DeleteCriticalSection(&g_logCs);
    return 0;
}
```